

OE-EC604C — Object Oriented Programming

Semester VI • 3L:0T:0P • 3 Credits

Module 5 Exam Notes: Operator Overloading

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Fundamentals of Operator Overloading	2
1.1	General Rules and Syntax	2
2	Binary & Unary Operator Overloading	2
2.1	Unary Operator Overloading	2
2.2	Binary Operator Overloading	3
2.3	Stream Insertion (<<) and Extraction (>>) Overloading	4
3	Type Conversions	4
3.1	Basic Type to Class Type	5
3.2	Class Type to Basic Type	5

1. Fundamentals of Operator Overloading

Operator overloading is a feature of compile-time polymorphism that allows user-defined types (classes) to interact using standard C++ operators.

1.1 General Rules and Syntax

When overloading operators in C++, the following rules must be observed:

- Only existing operators can be overloaded. New operators (like `**` or `<>`) cannot be created.
- The precedence and associativity of operators remain unchanged.
- Overloaded operators must have at least one operand of user-defined type (class/enum).
- Original syntax templates (e.g., number of operands) must be maintained.

List of Operators that Cannot be Overloaded

The following operators are strictly restricted from being overloaded:

1. Class member access operator (`.`)
2. Pointer-to-member operator (`.*`)
3. Scope resolution operator (`::`)
4. Ternary conditional operator (`?:`)
5. Sizeof operator (`sizeof`)

Operator Overloading Signature Syntax

As Member Function: `Return_Type operator OP (Arguments) {...}` (1)

As Friend Function: `friend Return_Type operator OP (Arguments) {...}` (2)

2. Binary & Unary Operator Overloading

Operators are overloaded differently based on whether they are unary (one operand) or binary (two operands), and whether member or friend functions are used.

2.1 Unary Operator Overloading

Unary operators operate on a single operand.

- **Member Function implementation:** Takes **zero** arguments (the operand is implicitly passed via the `this` pointer).

- **Friend Function implementation:** Takes **one** argument explicitly.
- **Increment/Decrement Ambiguity:** To distinguish between prefix (e.g., ++obj) and postfix (e.g., obj++), C++ uses a dummy `int` parameter in the postfix signature.

Code Example: Prefix and Postfix operator overloading

```
#include <iostream>
class Counter {
private:
    int value;
public:
    Counter(int v = 0) : value(v) {}

    // Prefix ++ (e.g., ++c)
    Counter operator++() {
        ++value;
        return *this; // Returns incremented object
    }

    // Postfix ++ (e.g., c++) -> Note the dummy 'int' parameter
    Counter operator++(int) {
        Counter temp = *this; // Save original state
        value++; // Increment
        return temp; // Return original state
    }

    int getValue() { return value; }
};
```

2.2 Binary Operator Overloading

Binary operators operate on two operands (e.g., `a + b`).

- **Member Function implementation:** Takes **one** argument explicitly. The left-hand operand is the calling object, and the right-hand operand is passed as the argument.
- **Friend Function implementation:** Takes **two** arguments explicitly.

Code Example: Relational operator overloading for custom strings

```

#include <iostream>
#include <cstring>
class String {
private:
    char *str;
public:
    String(const char *s) {
        str = new char[strlen(s) + 1];
        strcpy(str, s);
    }
    ~String() { delete[] str; }

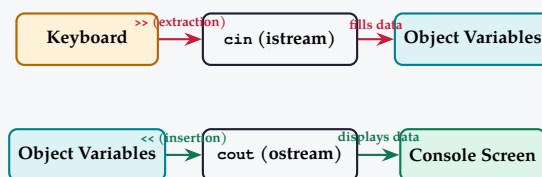
    // Overload == operator
    bool operator==(const String &s) {
        return strcmp(str, s.str) == 0;
    }

    // Overload < operator
    bool operator<(const String &s) {
        return strcmp(str, s.str) < 0;
    }
};

```

2.3 Stream Insertion (<<) and Extraction (>>) Overloading

Stream operators must be overloaded as **friend functions** because the left-hand operand is a stream object (like `std::ostream`), not an instance of our class.

Stream I/O Data Flow Block Diagram**3. Type Conversions**

C++ allows custom type conversions between primitive types and class objects.

3.1 Basic Type to Class Type

Handled automatically using a **parameterized constructor** inside the target class that accepts the basic type as an argument.

```
Time t1 = 95; // Invokes constructor Time(int) to convert minutes (3)
```

3.2 Class Type to Basic Type

Accomplished by defining a **casting operator function** (conversion function) inside the class.

Rules for Conversion Functions:

- It must be a class member function.
- It must not specify any return type (the return type is implicit from the operator's name).
- It must not accept any arguments.

```
Syntax:  operator Basic_Type () { return value; } (4)
```

Code Example: Basic-to-Class and Class-to-Basic conversions

```
#include <iostream>
class Time {
private:
    int hours, minutes;
public:
    Time() : hours(0), minutes(0) {}

    // Basic to Class Conversion (Constructor)
    Time(int mins) {
        hours = mins / 60;
        minutes = mins % 60;
    }

    // Class to Basic Conversion (Casting Operator)
    operator int() {
        return (hours * 60) + minutes;
    }

    void display() {
        std::cout << hours << "h " << minutes << "m" << std::endl;
    }
};

int main() {
    // Basic -> Class
    Time t1 = 85; // Converts 85 (int) to 1h 25m (Time)
    t1.display(); // Prints: 1h 25m

    // Class -> Basic
    int totalMins = t1; // Converts Time to int using operator int()
    std::cout << "Total minutes = " << totalMins << std::endl; // 85
    return 0;
}
```

Quick Revision — Module V

Operator Overloading | OE-EC604C

Key Overloading Signatures

Unary (Member)	<code>Counter operator++() { /* prefix */ }</code>
Unary Postfix (Member)	<code>Counter operator++(int) { /* dummy int parameter */ }</code>
Binary (Member)	<code>Complex operator+(const Complex &c) { /* 1 arg */ }</code>
Binary (Friend)	<code>friend Complex operator+(Complex &c1, Complex &c2); // 2 args</code>
Stream Insertion	<code>friend ostream& operator<<(ostream& out, const MyClass &obj);</code>
Casting Function	<code>operator float() { return data; } // No return type specified</code>

One-Line Definitions

- **Operator Overloading:** Assigning new functionalities to C++ operators when applied to class instances.
- **Stream Extraction (»):** An overloaded operator used to extract formatted values from an input stream.
- **Stream Insertion («):** An overloaded operator used to insert formatted values into an output stream.
- **Conversion Operator:** A special class member function used to convert an object into another data type.
- **Prefix Operator:** An increment or decrement operator executed before evaluation of the expression.
- **Postfix Operator:** An increment or decrement operator executed after evaluation of the expression.
- **Basic to Class Conversion:** Converting standard types to user classes via parameterized constructors.
- **Class to Basic Conversion:** Converting classes to standard types via overloaded casting operators.
- **This Pointer:** An implicit pointer inside member functions that points to the calling object instance.

Frequently Asked / Viva Questions**Q1. Which operators cannot be overloaded in C++? Why?**

Operators like `.`, `.*`, `::`, `?:`, and `sizeof` cannot be overloaded to protect C++ syntax structures and core language bindings from corruption.

Q2. Why does a postfix overloaded operator require a dummy `int` parameter?

The compiler uses the `int` parameter strictly as a signature flag to distinguish postfix calls (`obj++`) from prefix calls (`++obj`).

Q3. Why must stream insertion (`<<`) and extraction (`>>`) operators be overloaded as friend functions?

The left-hand operand is `ostream` or `istream`, not our class object. Member functions require the left operand to be the class itself, so a friend function is mandatory.

Q4. What is the difference in arguments between member and friend function operator overloading?

For binary operators: member functions take 1 argument (left operand is implicit); friend functions take 2 arguments explicitly. For unary operators: member functions take 0 arguments; friend functions take 1.

Q5. Can we change the basic behaviors of operators (like changing `+` to subtract)?

Yes, syntactically possible, but highly discouraged as it violates design clarity and makes code unreadable.

Q6. What are the restrictions on type conversion functions?

They must be non-static class member functions, cannot have arguments, and must not specify a return type.

Q7. Explain conversion from one class type to another class type.

This can be done using: (1) A constructor in the destination class that accepts the source class, or (2) A casting operator inside the source class that returns the destination class.

Q8. Can we overload the assignment operator (`=`) as a friend function?

No. C++ rules mandate that `=`, `()`, `[]`, and `->` must be overloaded *only* as member functions to prevent compiler syntax conflicts.

Q9. What is the return type of stream insertion (`<<`) and extraction (`>>`) operators?

They return a reference to the stream object (`ostream&` or `istream&`). This allows chaining (e.g., `cout << a << b;`).

Q10. Does operator overloading change the execution speed of a program?

No. Operator overloading is resolved at compile time (static binding), so it has zero runtime overhead compared to equivalent function calls.

Q11. Is it possible to overload operators for standard types (like overloading `+` for two integers)?

Frequently Asked / Viva Questions (contd.)

No. At least one operand must be a user-defined class or structure to prevent overriding standard arithmetic logic.

Q12. Can we use default arguments in overloaded operators?

No, default arguments are not permitted in overloaded operators because they could conflict with the operator's expected arity.

Comparison: Member Functions vs. Friend Functions for Overloading

Feature	Member Function Overloading	Friend Function Overloading
Implicit Operand	Passes the left-hand operand implicitly via the <code>this</code> pointer.	No implicit operand. All operands must be explicitly declared in arguments.
Left Operand Rule	Left operand must strictly be an object of the defining class.	Left operand can be of any type (e.g., stream, primitive, other class).
Number of Arguments	Binary: 1 argument; Unary: 0 arguments.	Binary: 2 arguments; Unary: 1 argument.
Class Encapsulation	Maintains strict encapsulation boundaries.	Grants external function access (minor encapsulation bypass).
Special Operators	Mandatory for operators: <code>=</code> , <code>[]</code> , <code>()</code> , <code>-></code> .	Not allowed for: <code>=</code> , <code>[]</code> , <code>()</code> , <code>-></code> .